

 Project Title :

IoT Sensor Data Archival and Filtering Using HDFS & HBase

 Project Objectives :

- To archive large-scale IoT sensor data using HDFS-inspired distributed storage for scalable data management.
- To enable efficient access and organization of IoT data using an HBase-style column-family data model.
- To process and filter IoT sensor data using Python to identify high energy consumption and low transmission efficiency sensors.
- To demonstrate data filtering at multiple storage layers, including both HDFS and HBase.
- To showcase the practical application of Big Data technologies in IoT data management and performance optimization.

```
# 📁 Upload Dataset to Google Colab
# This step allows the user to upload a local CSV file
# into the Google Colab environment for further processing.
```

```
from google.colab import files
uploaded = files.upload()
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving cde_ipaas_dataset.csv to cde_ipaas_dataset (1).csv

```
# 📊 Load IoT Sensor Dataset
# This step reads the uploaded CSV file into a Pandas DataFrame.
# The DataFrame allows easy data exploration, filtering, and analysis
```

```
import pandas as pd
```

```
df = pd.read_csv("cde_ipaas_dataset.csv")
df.head()
```

	query_id	sensor_type	data_size_bytes	quantity	duration	energy_consumption	data_yield	hypervolume_value	transmission_e
0	1	PulseOximeter	18	12	261	0.240395	152.275251	0.933088	
1	2	PulseOximeter	18	12	375	0.228580	685.799626	0.528206	
2	3	PulseOximeter	18	12	351	0.459875	141.999097	0.986878	
3	4	BloodPressure	14	12	507	0.444216	984.907797	0.733381	
4	5	BodyTemperature	12	12	746	0.158546	953.996984	0.982816	

```
# 🗄️ Store Dataset in HDFS (Simulated)
# This command creates an HDFS-like directory structure in Google Colab
# and copies the IoT dataset into it to simulate data storage in HDFS.
```

```
!mkdir -p /content/hdfs/iot_archive
!cp cde_ipaas_dataset.csv /content/hdfs/iot_archive/
```

```
# 📂 Access Dataset from HDFS (Simulated)
# This step reads the IoT dataset from the HDFS-like directory
# created earlier in Google Colab and loads it into a Pandas DataFrame
# for further processing and analysis.
```

```
hdfs_df = pd.read_csv("/content/hdfs/iot_archive/cde_ipaas_dataset.csv")
hdfs_df.head()
```

	query_id	sensor_type	data_size_bytes	quantity	duration	energy_consumption	data_yield	hypervolume_value	transmission_e
0	1	PulseOximeter	18	12	261	0.240395	152.275251	0.933088	
1	2	PulseOximeter	18	12	375	0.228580	685.799626	0.528206	
2	3	PulseOximeter	18	12	351	0.459875	141.999097	0.986878	
3	4	BloodPressure	14	12	507	0.444216	984.907797	0.733381	
4	5	BodyTemperature	12	12	746	0.158546	953.996984	0.982816	

```
# 🔍 Apply Filter on Data Stored in HDFS (Simulated)
# This step filters the IoT dataset accessed from the HDFS-like storage.
# Condition applied:
# 1. Energy consumption greater than 0.4
# 2. Transmission efficiency less than 3
# The filtered data represents specific IoT sensor readings
# that meet the defined criteria.
```

```
filtered_hdfs = hdfs_df[
    (hdfs_df["energy_consumption"] > 0.4) &
    (hdfs_df["transmission_efficiency"] < 3)
]
```

```
# Display the first few records of the filtered dataset

filtered_hdfs.head()
```

	query_id	sensor_type	data_size_bytes	quantity	duration	energy_consumption	data_yield	hypervolume_value	transmission_efficiency
2	3	PulseOximeter	18	12	351	0.459875	141.999097	0.986878	
5	6	ECG	135	12	87	0.446875	114.369627	0.615447	
6	7	PulseOximeter	18	12	268	0.545659	130.949669	0.954660	
9	10	PulseOximeter	18	12	355	0.638110	929.686812	0.544246	
10	11	PulseOximeter	18	12	340	0.449810	344.214129	0.914369	

```
# Store Filtered Data in HBase (Simulated)
# Since native HBase is not available in Google Colab,
# we simulate an HBase table using a Python dictionary.
#
# Structure:
# - Row Key      → query_id
# - Column Family → meta, metrics
# - Columns      → sensor_type, energy_consumption, data_yield,
#                  transmission_efficiency

hbase_table = {}

for _, row in filtered_hdfs.iterrows():

    # Use query_id as the HBase row key

    row_key = str(row["query_id"])

    # Insert data into simulated HBase table

    hbase_table[row_key] = {
        "meta": {
            "sensor_type": row["sensor_type"]
        },
        "metrics": {
            "energy_consumption": row["energy_consumption"],
            "data_yield": row["data_yield"],
            "transmission_efficiency": row["transmission_efficiency"]
        }
    }

# Display first 3 rows from the simulated HBase table

list(hbase_table.items())[:3]
```

```
[('3',
 {'meta': {'sensor_type': 'PulseOximeter'},
  'metrics': {'energy_consumption': 0.45987487454373,
             'data_yield': 141.99909689225387,
             'transmission_efficiency': 0.8681647438294037}}),
 ('6',
 {'meta': {'sensor_type': 'ECG'},
  'metrics': {'energy_consumption': 0.4468748522859245,
             'data_yield': 114.36962699819276,
             'transmission_efficiency': 1.810490015200696}}),
 ('7',
 {'meta': {'sensor_type': 'PulseOximeter'},
  'metrics': {'energy_consumption': 0.5456592191001431,
             'data_yield': 130.94966900369656,
             'transmission_efficiency': 0.8548638089379481}})]
```

```
# Apply Filter on Data Stored in HBase (Simulated)
# This step filters records from the simulated HBase table.
# Condition applied:
# - Data yield greater than 800
# This represents server-side style filtering on HBase data.

filtered_hbase = {
    k: v for k, v in hbase_table.items()
    if v["metrics"]["data_yield"] > 800
}
# Display the first 3 filtered HBase records

list(filtered_hbase.items())[:3]
```

```
[('10',
 {'meta': {'sensor_type': 'PulseOximeter'},
  'metrics': {'energy_consumption': 0.6381099809299766,
             'data_yield': 929.6868115208052,
             'transmission_efficiency': 2.2336145272305257}}),
 ('21',
 {'meta': {'sensor_type': 'BloodPressure'},
  'metrics': {'energy_consumption': 0.884314531168946,
             'data_yield': 823.304869209203,
             'transmission_efficiency': 0.920589594827552}}),
 ('37',
 {'meta': {'sensor_type': 'BloodPressure'},
  'metrics': {'energy_consumption': 0.7303220469754942,
             'data_yield': 861.9950280144753,
             'transmission_efficiency': 2.2402956860540084}})]
```



Project Conclusion

This project successfully demonstrates how IoT sensor data can be efficiently archived, accessed, and filtered using big data concepts inspired by the Hadoop ecosystem. By leveraging HDFS for scalable data storage and HBase-style column-family modeling for real-time access, the system effectively handles large volumes of sensor data.

Despite execution constraints in Google Colab, the project preserves the architectural integrity of distributed storage and NoSQL processing systems. The filtering techniques applied help identify high-energy-consuming and low-efficiency sensors, which is critical for optimizing IoT networks and reducing operational costs.

Overall, this project highlights the practical application of Big Data technologies in IoT data management, making it highly relevant for real-world use cases such as sensor monitoring, performance optimization, and data archival systems.